

فاز سوم

کیوان ثاقب مجیدزاده

موضوع برنامه

هدف این برنامه بررسی هم‌ارزی معنایی دو فرمول گزاره‌ای است. دو فرمول φ و ψ زمانی هم‌ارز هستند که تحت تمام انتساب‌های ممکن برای اجتماع متغیرهای دو فرمول، مقدار یکسانی داشته باشند:

$$\varphi \equiv \psi.$$

اگر دو فرمول در تمام انتساب‌ها مقدار یکسانی داشته باشند، برنامه Equivalent چاپ می‌کند. اگر حداقل در یک انتساب مقدار آن‌ها متفاوت باشد، خروجی Not Equivalent است و اولین انتساب نقض‌کننده نیز چاپ می‌شود. کد برای بررسی هم‌زمان تمام انتساب‌ها از عددهای صحیح به عنوان bitset استفاده می‌کند. هر بیت، مقدار فرمول را در یک انتساب مشخص نشان می‌دهد. اختلاف دو فرمول نیز با عملگر بیتی XOR محاسبه می‌شود.

تعریف هم‌ارزی معنایی

اگر مجموعه تمام متغیرهای ظاهر شده در دو فرمول را با X نشان دهیم، هم‌ارزی معنایی به صورت زیر تعریف می‌شود:

$$\varphi \equiv \psi \iff \forall v : X \rightarrow \{0, 1\}, \quad v(\varphi) = v(\psi).$$

برای بررسی هم‌ارزی باید اجتماع متغیرهای هر دو فرمول در نظر گرفته شود. حتی اگر یک متغیر فقط در یکی از فرمول‌ها ظاهر شده باشد، باید هنگام ساخت انتساب‌ها و چاپ مثال نقض حضور داشته باشد. اگر دو فرمول هم‌ارز نباشند، انتسابی که در آن مقدار دو فرمول متفاوت است یک مثال نقض برای هم‌ارزی محسوب می‌شود.

نمادهای قابل قبول

نماد متنی	معنی
a تا z	متغیر گزاره‌ای تک حرفی
T	ثابت درست
F	ثابت نادرست
!A	نقیض A
(A&B)	عطف $A \wedge B$
(A B)	فصل $A \vee B$
(A->B)	شرطی $A \rightarrow B$
(A<->B)	دوشرطی $A \leftrightarrow B$

متغیرهای گزاره‌ای فقط حروف کوچک انگلیسی هستند. هر زیرفرمول دوتایی باید داخل یک جفت پرانتز قرار گیرد. نقیض مستقیماً پیش از زیرفرمول بعدی نوشته می‌شود و پرانتز جداگانه نمی‌گیرد. ورودی نیز نباید فاصله اضافی داشته باشد.

قالب ورودی

ورودی دقیقاً شامل دو خط است:

۱. خط اول شامل فرمول φ ؛

۲. خط دوم شامل فرمول ψ .

برای نمونه:

$\neg(p \wedge q)$
 $(\neg p \vee \neg q)$

قالب خروجی

اگر دو فرمول هم‌ارز باشند، خروجی تنها یک خط است:

Equivalent

اگر دو فرمول هم‌ارز نباشند، خط اول Not Equivalent و خط دوم اولین انتساب نقض‌کننده است:

Not Equivalent
 $p = \text{False} \quad q = \text{True}$

متغیرها در خط دوم به ترتیب الفبایی چاپ می‌شوند. برای هر متغیر، False پیش از True در نظر گرفته می‌شود.

ایده اصلی استفاده از bitset

اگر در اجتماع متغیرهای دو فرمول، k متغیر متمایز وجود داشته باشد، تعداد تمام انتساب‌های ممکن برابر است با:

$$V = 2^k.$$

برنامه برای هر فرمول یک عدد صحیح با V بیت می‌سازد. بیت شماره i ، مقدار فرمول را در انتساب شماره i نشان می‌دهد:

وضعیت فرمول در انتساب شماره i	bit_i
درست	1
نادرست	0

متغیر `full_mask` عددی است که تمام V بیت معتبر آن برابر یک هستند:

$$\text{full_mask} = 2^V - 1.$$

این عدد نتیجه ثابت T در تمام انتساب‌ها را نمایش می‌دهد. عدد صفر نیز نتیجه ثابت F است.

ترتیب انتساب‌ها

متغیرها ابتدا به ترتیب الفبایی مرتب می‌شوند. سپس برای هر متغیر، `False` پیش از `True` قرار می‌گیرد. برای متغیرهای p و q ترتیب انتساب‌ها به صورت زیر است:

شماره بیت	p	q
۰	False	False
۱	False	True
۲	True	False
۳	True	True

این ترتیب تضمین می‌کند که کم‌ارزش‌ترین بیت یک در ماسک اختلاف، همان اولین مثال نقض مورد انتظار باشد.

ساختار کلی اجرای برنامه

مراحل اجرای برنامه به ترتیب زیر است:

۱. خواندن دو فرمول در تابع `main`؛
۲. استخراج اجتماع متغیرهای دو فرمول؛

۳. مرتب کردن متغیرها به ترتیب الفبایی؛
۴. ساخت bit mask هر متغیر و مقدار full_mask؛
۵. تجزیه و ارزیابی هر دو فرمول در تمام انتساب‌ها؛
۶. محاسبه ماسک اختلاف با عملگر XOR؛
۷. چاپ Equivalent در صورت صفر بودن ماسک اختلاف؛
۸. یافتن و چاپ اولین مثال نقض در غیر این صورت.

کلاس FormulaParser

کلاس FormulaParser متن فرمول را از چپ به راست تجزیه و هم‌زمان ارزیابی می‌کند. این کلاس درخت جداگانه‌ای برای فرمول نمی‌سازد. هر فراخوانی parse_formula یک عدد صحیح برمی‌گرداند که نتیجه زیرفرمول فعلی در تمام انتساب‌ها است.

ویژگی‌های کلاس عبارت‌اند از:

ویژگی	توضیح
text	متن فرمول ورودی
variable_masks	نگاشت هر متغیر به bit mask آن
full_mask	عددی که تمام بیت‌های معتبر آن برابر یک هستند
position	محل فعلی خواندن متن فرمول

متد parse ابتدا parse_formula را فراخوانی می‌کند. پس از ارزیابی فرمول نیز بررسی می‌شود که همه نویسه‌های ورودی مصرف شده باشند. اگر نویسه اضافی باقی مانده باشد، خطای ValueError ایجاد می‌شود.

تجزیه و ارزیابی فرمول

متد parse_formula حالت‌های زیر را پردازش می‌کند:

- متغیر: bit mask آن متغیر برگردانده می‌شود.
- ثابت T: مقدار full_mask برگردانده می‌شود.
- ثابت F: مقدار صفر برگردانده می‌شود.
- نقیض: نتیجه زیرفرمول بعدی در محدوده full_mask معکوس می‌شود.
- فرمول دوتایی: طرف چپ، عملگر و طرف راست خوانده و نتیجه با متد apply_operator محاسبه می‌شود.
- نقیض با عملگر XOR و full_mask محاسبه می‌شود:

$$\neg A = \text{full_mask} \wedge A.$$

این کار فقط بیت‌های مربوط به انتساب‌های معتبر را معکوس می‌کند.

تشخیص عملگر

متد `parse_operator` عملگرهای بلندتر را زودتر بررسی می کند. بنابراین ترتیب بررسی به صورت زیر است:

<->
->
&
|

اگر هیچ کدام از عملگرها در محل فعلی پیدا نشوند، خطای `ValueError` ایجاد می شود.

اجرای عملگرهای منطقی

متد `apply_operator` عملگرها را روی دو `bitset` اجرا می کند:

عملیات بیتی	رابطه منطقی	عملگر
<code>left & right</code>	$A \wedge B$	<code>&</code>
<code>left right</code>	$A \vee B$	<code> </code>
<code>(full_mask ^ left) right</code>	$\neg A \vee B$	<code>-></code>
<code>full_mask ^ (left ^ right)</code>	$\neg(A \oplus B)$	<code><-></code>

برای عملگر دوشروطی، ابتدا XOR بیت های متفاوت دو طرف را مشخص می کند. سپس نتیجه در محدوده `full_mask` معکوس می شود تا بیت هایی که دو طرف در آنها برابرند، مقدار یک بگیرند.

تابع collect_variables

این تابع هر دو فرمول را نویسه به نویسه بررسی می کند. تمام حروف کوچک انگلیسی در یک set قرار می گیرند تا تکرارها حذف شوند. خروجی تابع sorted، اجتماع متغیرهای دو فرمول را به ترتیب الفبایی برمی گرداند. استفاده از اجتماع متغیرها مهم است. اگر متغیری فقط در یکی از دو فرمول ظاهر شده باشد، همچنان می تواند در تعیین هم ارزی مؤثر باشد و باید در مثال نقض چاپ شود.

تابع build_variable_masks

این تابع bit mask متغیرها و مقدار full_mask را می سازد. اگر تعداد متغیرها k باشد:

$$\text{valuation_count} = 2^k,$$

$$\text{full_mask} = 2^{\text{valuation_count}} - 1.$$

برای متغیر قرار گرفته در موقعیت i ، اندازه هر بلوک متوالی از مقادیر False یا True برابر است با:

$$\text{block_size} = 2^{k-i-1}.$$

سپس یک بلوک شامل بیت های یک ساخته می شود:

$$\text{ones_block} = 2^{\text{block_size}} - 1.$$

حلقه داخلی این بلوک را در محل هایی قرار می دهد که مقدار متغیر True است:

```
for start in range(
    block_size,
    valuation_count,
    2 * block_size,
):
    variable_mask |= ones_block << start
```

برای دو متغیر p و q، ماسک ها از بیت کم ارزش به پر ارزش به صورت زیر هستند:

متغیر	ترتیب بیت ها از کم ارزش به پر ارزش
p	0011
q	0101

تابع evaluate_formula

این تابع یک نمونه از FormulaParser می‌سازد و متد parse را فراخوانی می‌کند. خروجی، یک عدد صحیح bitset است که مقدار فرمول را در تمام انتساب‌ها نگه می‌دارد.

تابع solve

تابع solve ابتدا خط‌های خالی را حذف می‌کند و بررسی می‌کند که ورودی دقیقاً شامل دو فرمول باشد. در غیر این صورت، خطای ValueError ایجاد می‌شود.

دو فرمول در متغیرهای first_formula و second_formula قرار می‌گیرند. سپس اجتماع متغیرهای آن‌ها استخراج می‌شود و bit mask ساخته می‌شوند.

هر دو فرمول با یک مجموعه مشترک از ماسک متغیرها ارزیابی می‌شوند:

```
first_result = evaluate_formula(...)
second_result = evaluate_formula(...)
```

استفاده از مجموعه مشترک ضروری است؛ زیرا بیت شماره i در هر دو نتیجه باید دقیقاً به یک انتساب اشاره کند.

محاسبه ماسک اختلاف

پس از ارزیابی دو فرمول، برنامه ماسک اختلاف را با XOR می‌سازد:

```
difference_mask = first_result ^ second_result
```

رفتار هر بیت XOR در جدول زیر نشان داده شده است:

مقدار فرمول اول	مقدار فرمول دوم	بیت اختلاف
False	False	0
False	True	1
True	False	1
True	True	0

اگر difference_mask صفر باشد، هیچ انتسابی وجود ندارد که مقدار دو فرمول در آن متفاوت باشد. بنابراین خروجی Equivalent است.

اگر ماسک اختلاف صفر نباشد، هر بیت یک، انتسابی را مشخص می‌کند که در آن دو فرمول مقدار متفاوتی دارند. در این حالت خروجی Not Equivalent است.

یافتن اولین مثال نقض

تابع `find_first_set_bit` کم‌ارزش‌ترین بیت یک در ماسک اختلاف را پیدا می‌کند:

$$\text{lowest_set_bit} = x \& (-x).$$

شماره آن بیت نیز با رابطه زیر محاسبه می‌شود:

$$\text{index} = \text{lowest_set_bit.bit_length}() - 1.$$

به دلیل ترتیب انتساب‌ها، این شماره همان اولین مثال نقضی است که `False` برای هر متغیر پیش از `True` بررسی می‌شود.

تابع `format_valuation`

این تابع شماره انتساب و فهرست مرتب‌شده متغیرها را دریافت می‌کند. برای متغیر قرارگرفته در موقعیت i ، بیت مربوط به آن با رابطه زیر خوانده می‌شود:

$$\text{value} = (\text{valuation_index} \gg (k - i - 1)) \& 1.$$

مقدار صفر به `False` و مقدار یک به `True` تبدیل می‌شود. سپس رشته‌هایی مانند `p=False` و `q=True` با یک فاصله به هم متصل می‌شوند.

تابع `main`

تابع `main` دو خط ورودی را با `input` می‌خواند:

```
first_formula = input().strip()
second_formula = input().strip()
```

سپس دو خط با `join` به `"\\n"` به یک رشته تبدیل و به `solve` ارسال می‌شوند. در پایان، مقدار برگشتی `solve` چاپ می‌شود.

اجرای نمونه اول: فرمول‌های هم‌ارز

ورودی:

$\neg(p \wedge q)$
 $(\neg p \vee \neg q)$

طبق قانون دمورگان داریم:

$$\neg(p \wedge q) \equiv \neg p \vee \neg q.$$

بنابراین بیت‌های نتیجه دو فرمول در تمام انتساب‌ها یکسان هستند و ماسک اختلاف برابر صفر می‌شود.
خروجی:

Equivalent

اجرای نمونه دوم: فرمول‌های غیرهم‌ارز

ورودی:

$(p \rightarrow q)$
 $(q \rightarrow p)$

در انتساب $p=\text{False}$ $q=\text{True}$ ، فرمول $p \rightarrow q$ درست و فرمول $q \rightarrow p$ نادرست است. این انتساب اولین مثال نقض بر اساس ترتیب تعیین شده است.
خروجی:

Not Equivalent
 $p=\text{False}$ $q=\text{True}$

پیچیدگی زمانی و حافظه

اگر تعداد متغیرهای متمایز در اجتماع دو فرمول را k در نظر بگیریم، تعداد انتساب‌ها برابر است با:

$$V = 2^k.$$

هر نتیجه میانی یک عدد صحیح با V بیت است. بنابراین عملیات بیتی با رشد k روی عددهای بزرگ‌تری اجرا می‌شوند. ساخت ماسک متغیرها نیز به فضایی متناسب با kV بیت نیاز دارد.

اگر مجموع طول دو فرمول را L در نظر بگیریم، بخش تجزیه نویسه‌های فرمول‌ها را به ترتیب بررسی می‌کنند؛ اما عملگرهای منطقی باید روی bitset‌هایی با V بیت اجرا شوند. در نتیجه، عامل اصلی رشد زمان و حافظه، تعداد نمایی انتساب‌ها است.

محدودیت‌های برنامه

- ورودی باید دقیقاً شامل دو فرمول غیرخالی باشد.
- نام هر متغیر فقط یک حرف کوچک انگلیسی از a تا z است.
- زیرفرمول‌های دوتایی باید کاملاً داخل پرانتز قرار گیرند.
- فرمول‌ها نباید فاصله اضافی داشته باشند.
- تعداد انتساب‌ها با تعداد متغیرهای متمایز به صورت نمایی رشد می‌کند.
- افزایش حد بازگشت به 10000 امکان پردازش فرمول‌های عمیق‌تر را فراهم می‌کند، اما فرمول بسیار عمیق همچنان می‌تواند منابع زیادی مصرف کند.